

GSPO：组序列策略优化

导论

一次 GSPO 更新可以概括为：

1. 旧策略对每道题生成 G 条回答
2. 验证器为每条完整回答计算奖励
3. 根据组内奖励计算相对优势
4. 计算每个词元的新旧策略概率比
5. 对词元对数概率比取平均
6. 指数化得到序列级概率比
7. 以完整回答为单位进行裁剪
8. 反向传播更新语言模型参数

$$\text{其中序列概率比的计算是 } s_i = \exp\left(\frac{1}{T_i} \sum_{t=1}^{T_i} \log \rho_{i,t}\right)$$

GRPO 对每个词元分别计算重要性比率，而 GSPO 认为奖励是针对整条回答给出的，因此重要性比率、裁剪和优化也应当以整条回答为单位。

GSPO 由 Qwen 团队提出，论文将其用于改善长序列和 MoE 模型强化学习中的训练稳定性。（<https://arxiv.org/pdf/2507.18071v2>）

对于第 i 条回答的第 t 个词元，GRPO 计算 $\rho_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$ ，然后对每一个词元分别进行裁剪 $\text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)$ ，虽然一条回答中的所有词元通常共享同一个回答级优势： $\hat{A}_{i,t} = \hat{A}_i$ ，但每个词元对应的概率比不同： $\rho_{i,1}, \rho_{i,2}, \dots, \rho_{i,|y_i|}$ ，所以每个词元会获得不同的梯度权重。

在数学推理训练中，奖励通常是针对整条回答给出的：

$$r(x, y_i) = \begin{cases} 1, & \text{最终答案正确,} \\ 0, & \text{最终答案错误.} \end{cases}$$

也就是说，奖励的对象是完整回答 y_i ，但是 GRPO 却在每个词元上分别计算重要性比 $\rho_{i,t}$ ，GSPO 论文认为，这会导致一种单位不匹配，即奖励单位为完整回答，重要性采样单位为单个词元。

尤其是在长回答中，大量不同的词元概率比会产生较高方差；这些噪声可能随着序列长度累积。论文将这一点视为 GRPO 在大模型、长序列和 MoE 强化学习中不稳定的重要原因。这里属于论文作者提出并通过实验支持的分析，而不是所有场景下都被证明的普遍结论。

GSPO 与 GRPO 一样，对同一道题 x 生成 G 条回答： $\{y_1, y_2, \dots, y_G\} \sim \pi_{\theta_{\text{old}}}(\cdot | x)$

获得对应奖励 $r_1, r_2, \dots, r_G$ ，然后计算组内相对优势 $\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon_{\text{num}}}$

$$\text{其中 } \mu_r = \frac{1}{G} \sum_{j=1}^G r_j; \sigma_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_r)^2}$$

因此，GSPO 仍然具有以下特征：

1. 不需要 Critic；
2. 同一道题生成多条回答；
3. 根据组内奖励计算相对优势；
4. 正优势回答提高概率；
5. 负优势回答降低概率。

它与 GRPO 的主要区别在策略概率比的计算方式。

假设回答 y_i 包含 T_i 个词元： $y_i = (y_{i,1}, y_{i,2}, \dots, y_{i,T_i})$ ，其中 $T_i = |y_i|$ ，自回归语言模型生成整条回答的概率为 $\pi_\theta(y_i | x) = \prod_{t=1}^{T_i} \pi_\theta(y_{i,t} | x, y_{i,<t})$ ，旧策略下的回答概率为

$$\pi_{\theta_{\text{old}}}(y_i | x) = \prod_{t=1}^{T_i} \pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})$$

因此，完整回答的新旧策略概率比为 $\frac{\pi_\theta(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)}$

将乘积展开 $\frac{\pi_\theta(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)} = \prod_{t=1}^{T_i} \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$ ，也就是所有词元概率比的乘积。

为什么需要长度归一化？

如果直接把所有词元概率比相乘，长回答的比值很容易变得极大或者极小。

例如，如果每个词元的概率比都是：1.01

对于长度为 100 的回答： $1.01^{100} \approx 2.705$

对于长度为 1000 的回答： $1.01^{1000} \approx 20959$

虽然单个词元只变化了约1%，但完整序列概率比却会随着长度指数增长。

GSPO 对序列概率比取 T_i 次方根： $s_i(\theta) = \left(\frac{\pi_\theta(y_i | x)}{\pi_{\theta_{\text{old}}}(y_i | x)} \right)^{\frac{1}{T_i}}$ ，也可以写成

$$s_i(\theta) = \exp \left(\frac{1}{T_i} \sum_{t=1}^{T_i} \log \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})} \right)$$

因此 GSPO 的序列概率比就是一条回答中所有词元概率比的几何平均值。长度归一化能够降低不同长度回答之间的数值差异，并避免序列概率比随着回答长度发生剧烈变化。

GSPO 的目标函数

$$\text{GSPO 的目标为 } \mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right]$$

其中：

G 是同一道题生成的回答数量；

$s_i(\theta)$ 是第 i 条回答的长度归一化序列概率比；

$\hat{A}_i$ 是第 i 条回答的组内相对优势；

ϵ 是序列级裁剪范围。

和 GRPO 对比，GRPO: $\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \min(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$;

GSPO: $\frac{1}{G} \sum_{i=1}^G \min(s_i \hat{A}_i, \text{clip}(s_i, 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$ ，整条回答只有一个概率比，并且只做一次序列级裁剪。

假设同一道数学题生成两条回答 $G=2$

两条回答的奖励分别为 $r_1 = 1, r_2 = 0$ ；组内平均奖励为 $\mu_r = \frac{1+0}{2} = 0.5$ ；组内标准差为 $\sigma_r = 0.5$

$$\text{所以 } \hat{A}_1 = \frac{1-0.5}{0.5} = 1; \hat{A}_2 = \frac{0-0.5}{0.5} = -1$$

第一条回答是正确回答，优势为正；第二条回答是错误回答，优势为负。

假设第一条回答包含三个词元： $y_1 = (y_{1,1}, y_{1,2}, y_{1,3})$ ，新旧策略概率如下：

词元	旧策略概率	新策略概率	词元概率比
$y_{1,1}$	0.20	0.30	1.50
$y_{1,2}$	0.40	0.36	0.90
$y_{1,3}$	0.30	0.33	1.10

第一条回答三个词元的概率比分别为

$$\rho_{1,1} = \frac{0.30}{0.20} = 1.50; \rho_{1,2} = \frac{0.36}{0.40} = 0.90; \rho_{1,3} = \frac{0.33}{0.30} = 1.10$$

完整序列的未归一化概率比为: $\frac{\pi_\theta(y_1 | x)}{\pi_{\theta_{\text{old}}}(y_1 | x)} = 1.50 \times 0.90 \times 1.10$

计算得到 $1.50 \times 0.90 \times 1.10 = 1.485$

因为回答长度为 $T_1 = 3$

所以 GSPO 的长度归一化序列概率比为: $s_1 = 1.485^{\frac{1}{3}} \approx 1.141$

也可以通过取对数计算 $s_1 = \exp\left(\frac{\log 1.50 + \log 0.90 + \log 1.10}{3}\right) \approx 1.141$

假设第二条回答也有三个词元: $y_2 = (y_{2,1}, y_{2,2}, y_{2,3})$ ，对应概率如下：

词元	旧策略概率	新策略概率	词元概率比
$y_{1,1}$	0.40	0.20	0.50
$y_{1,2}$	0.20	0.18	0.90
$y_{1,3}$	0.50	0.40	0.80

序列的未归一化概率比为 $0.50 \times 0.90 \times 0.80 = 0.36$

长度归一化后 $s_2 = 0.36^{\frac{1}{3}} \approx 0.711$

因此，两条回答的 GSPO 序列概率比分别为 $s_1 \approx 1.141$; $s_2 \approx 0.711$ 为了便于理解，假设 $\epsilon = 0.2$，裁剪区间为 $[1 - \epsilon, 1 + \epsilon] = [0.8, 1.2]$ 第一条回答具有 $\hat{A}_1 = 1$; $s_1 = 1.141$，不触发裁剪。 未裁剪项为 $s_1 \hat{A}_1 = 1.141 \times 1 = 1.141$; 裁剪项为 $\text{clip}(1.141, 0.8, 1.2) \times 1 = 1.141$; 最终贡献为 $\min(1.141, 1.141) = 1.141$ 值得注意的是，第一个词元的单独概率比为 $\rho_{1,1} = 1.50$，在 GRPO 中，这个词元可能会因为超过 1.2 而被单独裁剪。但在 GSPO 中，整条回答的平均变化只有: $s_1 = 1.141$，所以整条回答不会被裁剪。 这说明 GSPO 判断的是整条回答相对于旧策略是否变化过大，而不是某个单独词元是否变化过大。 第二条回答具有 $\hat{A}_2 = -1$; $s_2 = 0.711$，由于 $s_2 < 0.8$，所以整条回答触发下界裁剪。 未裁剪项为 $s_2 \hat{A}_2 = 0.711 \times (-1) = -0.711$ 裁剪后的概率比为 $\text{clip}(0.711, 0.8, 1.2) = 0.8$ 裁剪项为 $0.8 \times (-1) = -0.8$ 最终取较小值 $\min(-0.711, -0.8) = -0.8$ 这里整个回答被作为一个优化单位处理。 两条回答的平均贡献为 $\mathcal{J}_{\text{GSPO}} = \frac{1}{2}(1.141 - 0.8)$，因此 $\mathcal{J}_{\text{GSPO}} \approx 0.1705$，训练过程中需要最大化这个目标。
--

GSPO 是否不再对词元进行梯度更新?

不是。

GSPO 只是使用一个序列级概率比进行加权和裁剪，语言模型参数仍然通过每个词元的对数概率进行反向传播。

暂时忽略裁剪，第 i 条回答的目标为: $\mathcal{J}_i = s_i(\theta) \hat{A}_i$

其梯度可以写成 $\nabla_{\theta} \mathcal{J}_i = s_i(\theta) \hat{A}_i \nabla_{\theta} \log s_i(\theta)$

[对目标函数求梯度 $\nabla_{\theta} \mathcal{J}_i(\theta) = \nabla_{\theta} [s_i(\theta) \hat{A}_i]$

因为 $\hat{A}_i$ 被视为常数，所以可以提出 $\nabla_{\theta} \mathcal{J}_i(\theta) = \hat{A}_i \nabla_{\theta} s_i(\theta)$ ，接下来只需要处理 $\nabla_{\theta} s_i(\theta)$

由于序列概率比一定为正 $s_i(\theta) > 0$ ，所以可以对它取对数。根据链式法则 $\nabla_{\theta} \log s_i(\theta) = \frac{1}{s_i(\theta)} \nabla_{\theta} s_i(\theta)$ ，两边同时乘以 $s_i(\theta)$: $\nabla_{\theta} s_i(\theta) = s_i(\theta) \nabla_{\theta} \log s_i(\theta)$ ，代入前面的式子 $\nabla_{\theta} \mathcal{J}_i(\theta) = \hat{A}_i s_i(\theta) \nabla_{\theta} \log s_i(\theta)$ ，调整乘法顺序，就得到:

$$\nabla_{\theta} \mathcal{J}_i(\theta) = s_i(\theta) \hat{A}_i \nabla_{\theta} \log s_i(\theta)$$

由于 $\log s_i(\theta) = \frac{1}{T_i} \sum_{t=1}^{T_i} \log \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$ ，旧策略不参与求导，所以：

$$\nabla_\theta \mathcal{J}_i = \frac{s_i(\theta) \hat{A}_i}{T_i} \sum_{t=1}^{T_i} \nabla_\theta \log \pi_\theta(y_{i,t} | x, y_{i,<t})$$

这说明每个词元仍然参与梯度计算；每个词元的对数概率梯度仍然不同；但同一条回答中的所有词元共享统一的标量权重： $\frac{s_i(\theta) \hat{A}_i}{T_i}$ ，GRPO 则为不同词元使用

不同的权重 $\frac{\rho_{i,t}(\theta) \hat{A}_i}{T_i}$

因此，二者的区别可以概括为：

GRPO 对于不同词元使用不同的概率比权重；GSP 对于同一回答中的词元共享一个序列概率比权重。

论文认为，统一权重可以减少词元级概率比波动带来的训练噪声。

为什么 GSPO 的裁剪范围非常小？

在前面的数值例子中，为了便于理解，我们使用 $\epsilon = 0.2$ ，但原始 GSPO 实验使用的裁剪范围要小得多。

论文实验中，GSPO 左右裁剪宽度分别设置为 $\epsilon_{\text{low}} = 3 \times 10^{-4}$ ， $\epsilon_{\text{high}} = 4 \times 10^{-4}$

也就是序列概率比大约被限制在 $[0.9997, 1.0004]$

作为对照，论文中的 GRPO 左右裁剪宽度设置为：0.2 和 0.27

两种裁剪范围相差多个数量级，是因为二者的概率比定义不同：GRPO 使用单词元概率比；GSPO 使用整条回答的平均对数概率变化。

因此，不能把 GRPO 的 0.2 直接照搬成 GSPO 的裁剪参数。

GSPO 对 MoE 模型为什么更稳定？

在 MoE 模型中，同一个词元可能在不同前向传播中激活不同专家。

模型参数更新后，即使输入回答完全相同，专家路由也可能发生变化，进而使单个词元的新旧概率比剧烈波动。

论文在 Qwen3-30B-A3B-Base 上观察到，一次强化学习梯度更新后，对于相同 rollout 样本，大约有 10% 的专家激活与旧策略不同。作者认为，这会显著扰动 GRPO 的词元级概率比。

GRPO 对每个词元单独使用 $\rho_{i,t}$ ，因此某个位置的路由变化可能直接改变该词元的梯度权重和裁剪状态。

GSPO 使用整条回答的几何平均 $s_i = \exp\left(\frac{1}{T_i} \sum_{t=1}^{T_i} \log \rho_{i,t}\right)$ ，单个词元的异常变化会被其他词元平均，从而降低局部路由波动对整个优化目标的影响。

论文报告，GRPO 在其 MoE 实验中需要 Routing Replay 等额外策略维持正常收敛，而 GSPO 不需要该策略。

纯 GSPO 对整条回答使用一个优势 $\hat{A}_i$ ，但在多轮 Agent 或工具调用任务中，不

同词元、不同轮次可能需要不同的优势，因此论文还提出了 **GSPO-token**。

定义 $s_{i,t}(\theta) = \text{sg}[s_i(\theta)] \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\text{sg}[\pi_\theta(y_{i,t} | x, y_{i,<t})]}$ ，其中 $\text{sg}[\cdot]$ 表示 stop gradient，即只使用数值，但不允许梯度通过该部分传播。

因为 $\frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\text{sg}[\pi_\theta(y_{i,t} | x, y_{i,<t})]}$ 在数值上恒等于 1，所以 $s_{i,t}(\theta) = s_i(\theta)$ ，在数值和裁剪判断上，GSPO-token 仍使用序列级概率比。但它允许梯度在第 t 个词元位置单独传播，并允许使用不同的词元优势： $\hat{A}_{i,t}$

其目标可以写成：

$$\mathcal{J}_{\text{GSPO-token}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \min(s_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(s_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t}) \right]$$

当所有词元优势相同时 $\hat{A}_{i,t} = \hat{A}_i$ ，GSPO-token 与普通 GSPO 在目标数值、裁剪判断和理论梯度上等价，但 GSPO-token 可以进一步支持词元级、轮次级或动作级信用分配。

GSPO 的局限性

1. 信用分配仍然比较粗

一条回答中的所有词元共享同一个回答级优势： $\hat{A}_i$

如果回答前半部分正确、后半部分错误，普通 GSPO 很难判断具体是哪一步造成最终结果。GSPO-token 可以支持更细粒度的优势，但仍然需要外部方法构造可靠的词元级优势。

2. 整条回答一起被裁剪

如果一条回答的序列概率比超过裁剪范围，整条回答的策略梯度可能被限制。这可能会同时丢弃其中一部分原本有价值的词元信号。

3. 几何平均可能掩盖局部异常

例如一条回答中三个概率比为：[2.0, 1.0, 0.5]，它们的乘积为 $2.0 \times 1.0 \times 0.5 = 1$ ，所以序列概率比为 $s = 1^{\frac{1}{3}} = 1$ ，从整条回答看，新旧策略没有变化，但实际上第一个词元概率翻倍，第三个词元概率减半。

这是序列级方法在提高稳定性的同时付出的代价：它会弱化局部变化信息。

4. 裁剪参数不能直接继承 GRPO

GSPO 的裁剪范围与 GRPO 差异很大，需要重新调节。错误使用 0.2 等词元级裁剪参数，可能使 GSPO 的序列裁剪几乎不起作用。

总结

GRPO 使用 $\rho_{i,t} = \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$ ，每个词元有一个概率比。

GSPO 使用 $s_i = \left(\prod_{t=1}^{T_i} \rho_{i,t} \right)^{\frac{1}{T_i}}$ ，整条回答只有一个概率比。

GSPO 最重要的思想是既然奖励是赋给完整回答的，那么重要性采样、裁剪和优化也应尽量以完整回答为单位。

这样能够减少长序列中词元级概率比噪声的累积，尤其适合论文重点讨论的大规模长推理和 MoE 强化学习训练。原论文在 Qwen3-30B-A3B-Base 的实验中报告，GSPO 相比经调参并使用 Routing Replay 的 GRPO 获得了更稳定且更高效的训练曲线，并应用于 Qwen3 系列后训练。